



南開大學
Nankai University

控制系统设计与仿真 大作业报告

姓 名：殷家兴
学 号：2313666
专 业：自动化

人工智能学院

2026 年 6 月

目录

一 总体要求	1
二 程序实现效果	1
2.1 主窗口	1
2.2 基础设置窗口	5
2.3 参数设置窗口	6
2.4 保存与读取功能	7
三 Timer 定时器驱动机制实现	7
四 数值仿真算法实现	10
4.1 状态空间模型建立	10
4.2 PID 控制器实现	11
4.3 欧拉法实现	11
4.4 二阶龙格-库塔法实现	12
4.5 四阶龙格-库塔法实现	12
4.6 仿真状态更新	13

一 总体要求

根据太阳翼展开的高保真时变重力卸载控制系统模型，设计太阳翼展开位移控制系统，被控量为太阳翼位移，控制量为卷扬机构提供的拉力，采用 PID 控制器，对太阳翼位移进行控制。

二 程序实现效果

本系统采用 Visual Studio 2022 与 MFC 框架开发，实现了太阳翼展开时变重力卸载控制系统的可视化仿真。程序包含主窗口、基础设置窗口和参数设置窗口三部分，能够完成参数配置、实时数值仿真、曲线绘制以及太阳翼展开动画显示等功能。

2.1 主窗口

主窗口采用左右分区布局设计，左侧区域用于显示仿真结果曲线，右侧区域用于显示太阳翼展开动画。系统运行过程中能够实时更新控制量、误差及位移变化情况，为用户提供直观的仿真结果展示。

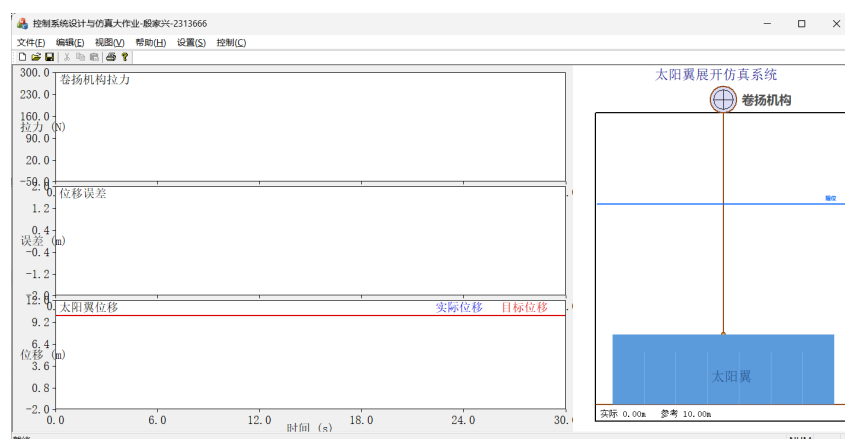


图 1: 程序主窗口

菜单栏与快捷键

程序主窗口顶部新增了“设置”和“控制”等菜单。其中，“设置”菜单包含基础设置和参数设置两个功能模块，用于完成系统显示参数和控制参数的配置；“控制”菜单包含开始、暂停和停止功能，用于控制仿真的运行状态。

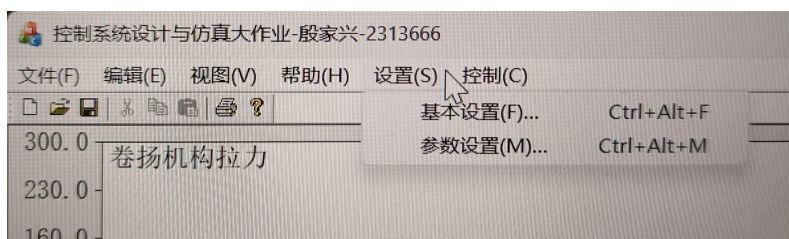


图 2: 设置菜单

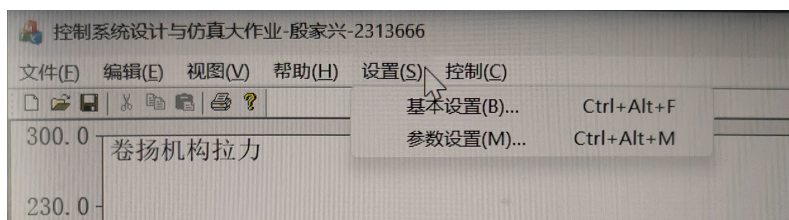


图 3: 控制菜单

菜单栏各选项均设置相应的快捷键。其中基础设置对应快捷键“Ctrl+Alt+F”，参数设置要求快捷键为“Ctrl+Alt+P”，但由于与系统快捷键设置冲突，因此这里改为“Ctrl+Alt+M”；开始仿真、暂停仿真和停止仿真分别对应“Alt+S”“Alt+P”和“Alt+T”。此外，在视图区域点击鼠标右键时可弹出控制快捷菜单，方便用户快速完成仿真控制操作。

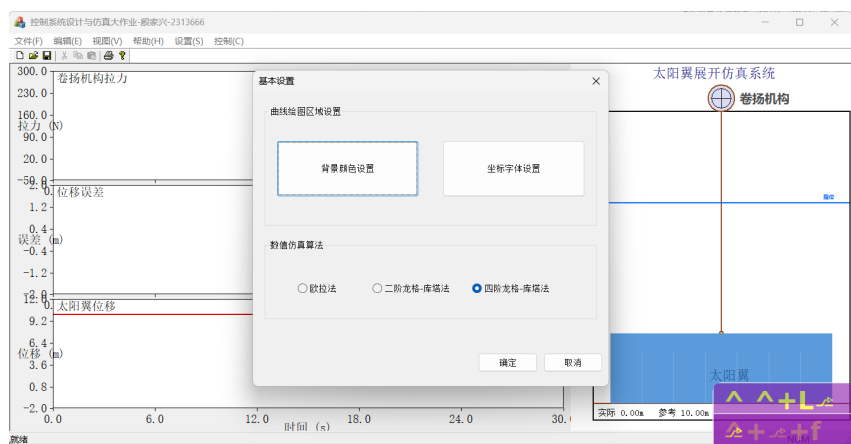


图 4: 基础设置窗口快捷键（“Ctrl+Alt+F”）

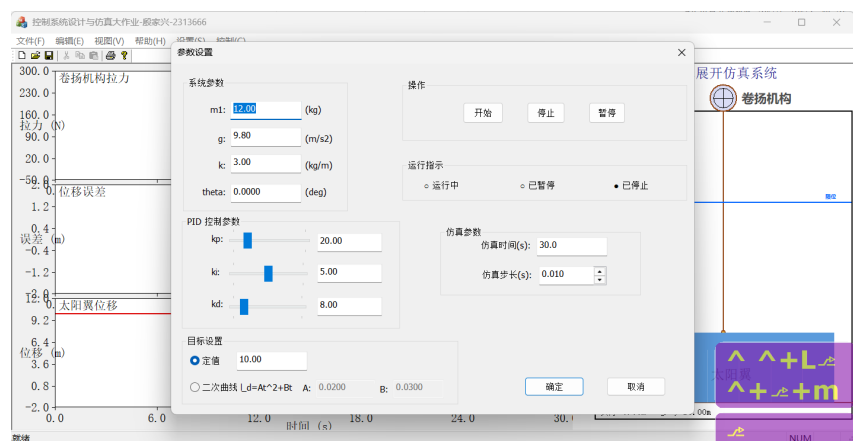


图 5: 参数设置窗口快捷键 (“Ctrl+Alt+M”)

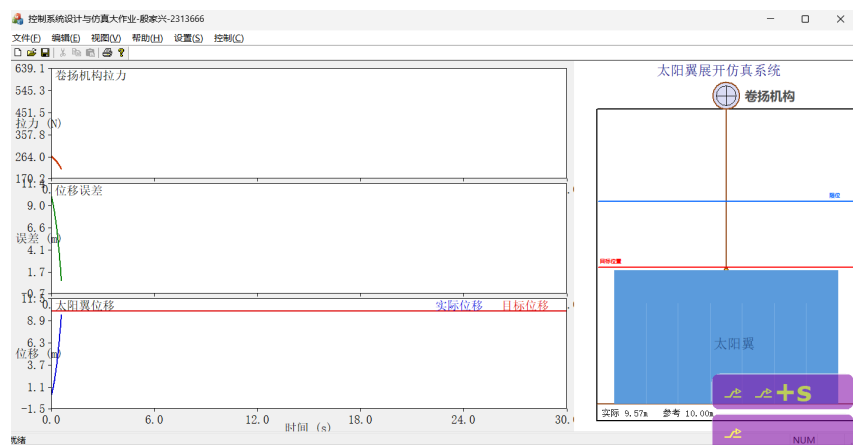


图 6: 开始快捷键 (“Alt+S”)

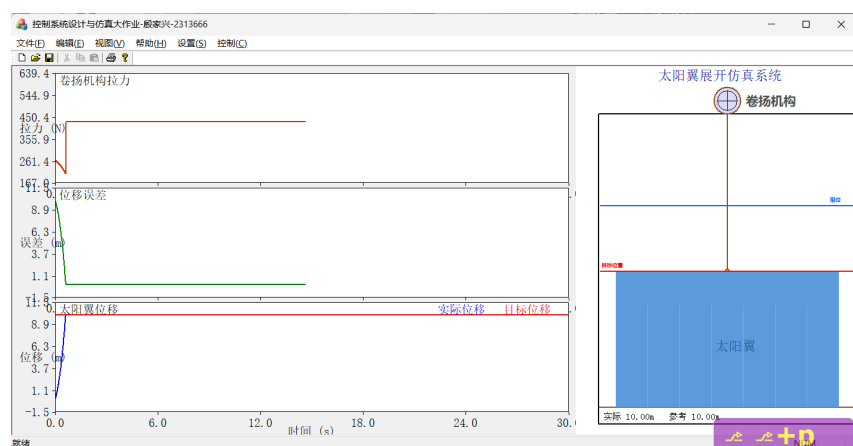


图 7: 暂停快捷键 (“Alt+P”)

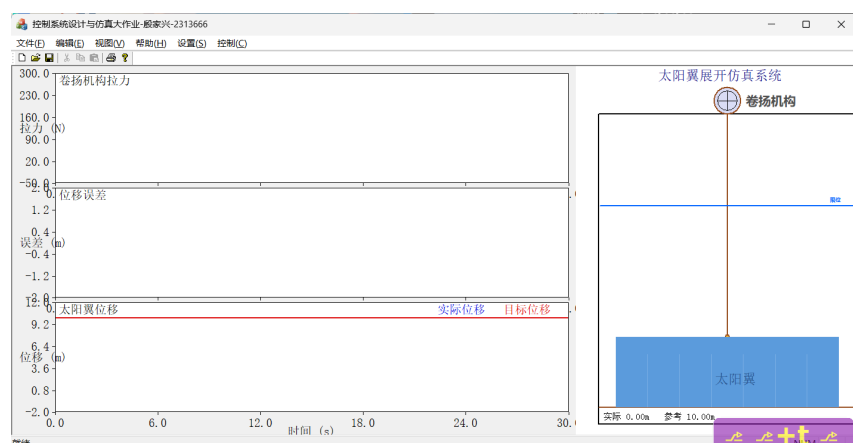


图 8: 停止快捷键 (“Alt+T”)

曲线绘制与点位标记

主窗口左侧区域用于实时显示系统仿真结果，绘制卷扬机构拉力曲线、位移误差曲线和太阳翼实际位移与目标位移对比曲线三幅曲线图。

拉力曲线反映 PID 控制器输出控制力的变化过程；误差曲线用于观察系统跟踪性能和稳态误差情况；位移曲线则用于比较实际位移与参考轨迹之间的跟踪效果。

系统采用双缓冲绘图技术进行曲线刷新，有效避免了界面闪烁现象，提高了绘图流畅性。曲线坐标轴范围根据仿真数据自动调整，能够保证数据完整显示。

当鼠标移动至曲线区域时，光标自动变为十字准星；单击鼠标左键后可显示当前坐标点的横纵坐标数值；双击鼠标左键可清除全部已标记点。该功能便于用户对关键时刻的系统响应数据进行精确读取和分析。

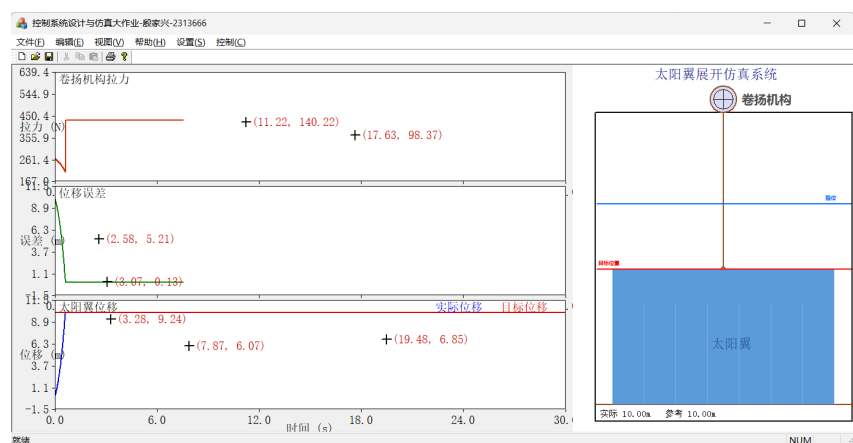


图 9: 曲线绘制与点位标记

展开动画

主窗口右侧区域实现了太阳翼展开过程的动画仿真。动画区域绘制了卷扬机构、钢丝绳以及太阳翼展开部分，通过读取数值仿真得到的实时位移数据，动态更新太阳翼的

位置状态。

随着控制系统输出拉力作用，太阳翼按照设定目标轨迹逐渐展开。动画中的太阳翼高度变化与数值计算结果保持同步，使用户能够直观观察系统控制过程。同时，动画区域实时显示当前位移值、目标位移值及运行状态等信息。

系统还设计了位置限位与报警机制。当太阳翼展开位移达到预设最大安全位置时，程序自动弹出报警提示窗口，并停止仿真运行，从而避免超出安全工作范围。

通过动画显示与曲线结果的结合，能够同时从数值和可视化两个角度观察控制系统性能。

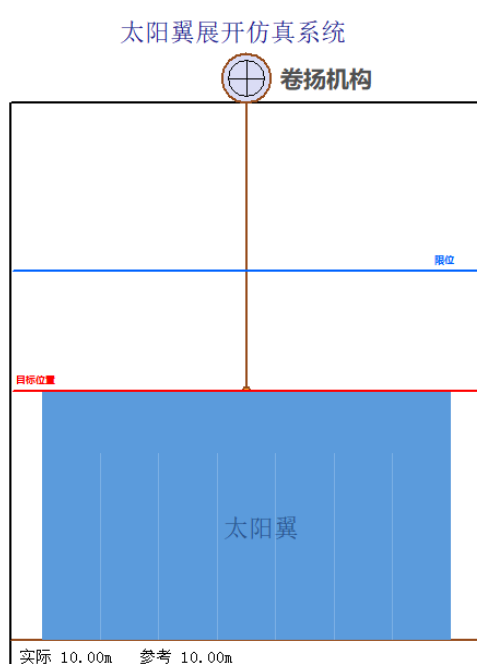


图 10: 太阳翼展开动画

2.2 基础设置窗口

基础设置窗口主要用于设置系统显示属性以及数值仿真算法。用户可根据实际需求修改曲线显示效果和数值积分方法。

在显示设置部分，用户可以自定义曲线绘图区背景颜色、文字颜色、字体类型以及字体大小。修改完成后，新的显示参数会立即应用到主窗口绘图区域，从而满足不同用户的界面显示需求。

在数值算法设置部分，系统提供欧拉法 (Euler Method)、二阶龙格—库塔法 (RK2)、四阶龙格—库塔法 (RK4) 和三种常用数值积分算法。

用户可通过单选按钮选择一种算法作为当前仿真计算方法。其中欧拉法计算速度较快，RK2 法兼顾速度与精度，而 RK4 法具有较高的计算精度，因此被设置为系统默认算法。



图 11: 基础设置对话框

展开动画

2.3 参数设置窗口

参数设置窗口用于配置系统动力学参数、PID 控制器参数、目标轨迹参数以及仿真参数，是整个系统的核心配置界面。



图 12: 参数设置对话框

系统动力学参数包括吊钩质量、太阳翼单位长度质量、重力加速度以及钢丝绳倾角等参数。程序对各参数设置了合法取值范围，当输入超出规定范围时会自动弹出错误提示信息，以保证仿真模型的合理性。

PID 控制器参数包括比例系数 (K_p)、积分系数 (K_i) 和微分系数 (K_d)。系统采用“滑块 + 输入框”的交互方式，用户既可以拖动滑块快速调整参数，也可以直接输入精确数值，实现控制器参数整定。

目标设置部分支持定值目标模式和二次曲线目标模式两种控制模式：

在定值模式下，用户直接输入目标位移值；在二次曲线模式下，通过设置参数 (A) 和 (B)，生成参考轨迹 $l_d = At^2 + Bt$ ，从而实现时变轨迹跟踪控制。

此外，窗口还提供仿真总时间和积分步长设置功能，并集成开始、暂停和停止按钮，方便用户在参数调试过程中快速观察不同参数对系统响应性能的影响。

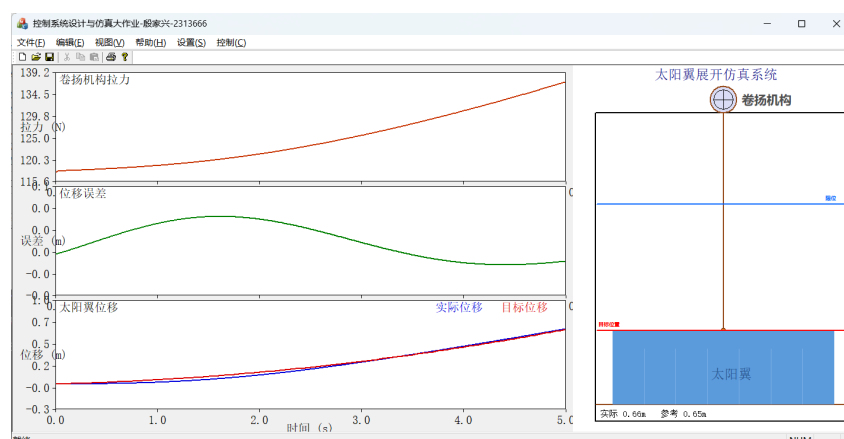


图 13: 二次曲线轨迹

2.4 保存与读取功能

为了提高系统的实用性和参数复用能力，程序实现了仿真参数的保存与读取功能。用户在完成基础设置、系统参数、PID 参数、目标轨迹参数以及仿真参数设置后，可通过“文件”菜单中的“保存”功能将当前配置保存到外部文件中；在后续使用过程中，可通过“读取参数”功能重新加载已保存的配置文件，无需重复输入各项参数。

这一功能的实现展示见视频。

三 Timer 定时器驱动机制实现

为了实现太阳翼展开过程的实时动态仿真，本系统采用 MFC 提供的 Timer 定时器机制作为仿真时钟源。定时器周期性触发消息，驱动系统按照设定步长进行数值积分计算，并实时刷新曲线和动画界面。

定时器消息映射

在视图类 C2313666yjsxView 中注册定时器消息：

```
BEGIN_MESSAGE_MAP(C2313666yjsxView, CView)
...
ON_WM_TIMER()
END_MESSAGE_MAP()
```

当定时器到达设定时间间隔时，Windows 系统自动向视图窗口发送 WM_TIMER 消息，并调用对应的 OnTimer() 函数。

仿真启动

用户点击“开始”菜单后执行：

```
void C2313666yjsxDoc::StartSimulation()
{
    if (!m_isRunning)
    {
        if (!m_isPaused)
        {
            ResetSimulation();
        }
        m_isRunning = true;
        m_isPaused = false;
    }
}
```

同时在视图类中启动定时器：

```
SetTimer(1, 10, NULL);
```

其中，定时器编号为 1，周期 10ms，回掉方式为消息驱动，即系统每 10 ms 更新一次仿真状态。

定时器响应过程

定时器触发后执行如下流程：

```
void C2313666yjsxView::OnTimer(UINT_PTR nIDEvent)
{
    C2313666yjsxDoc* pDoc = GetDocument();

    if(pDoc && pDoc->IsRunning())
    {
        pDoc->StepSimulation();

        Invalidate(FALSE);
    }

    CView::OnTimer(nIDEvent);
}
```

执行过程如下图所示。

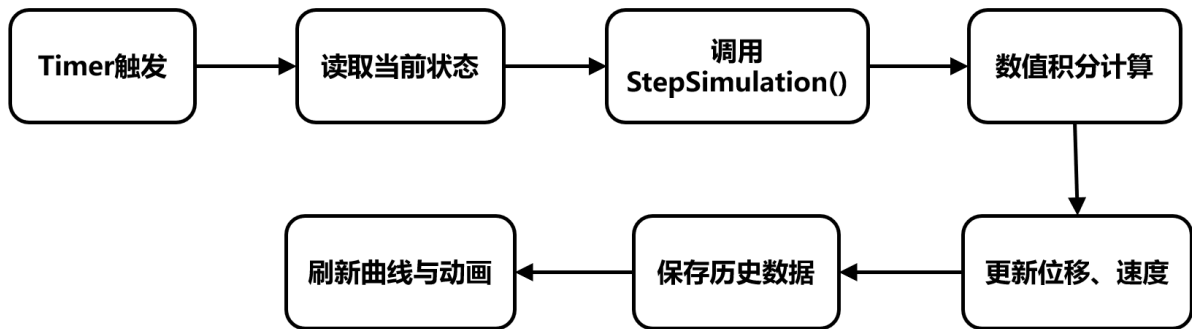


图 14: Timer 执行过程

暂停与停止

暂停功能:

```

void C2313666yjsxDoc::PauseSimulation()
{
    if (m_isRunning)
    {
        m_isRunning = false;
        m_isPaused = true;
    }
}
  
```

停止功能:

```

void C2313666yjsxDoc::StopSimulation()
{
    m_isRunning = false;
    m_isPaused = false;
    ResetSimulation();
}
  
```

暂停可以保留当前仿真状态，停止后重新初始化所有状态变量，定时器会停止刷新。

数据记录

每完成一次积分计算后，系统保存当前时刻数据:

```

DataPoint dp;
dp.t = m_currentTime;
dp.displacement = m_l;
dp.target = target;
  
```

```
dp.error = error;
dp.force = F_ctrl;
```

存入历史数据容器:

```
m_dataHistory.push_back(dp);
```

保存的数据用于拉力曲线、误差曲线和位移跟踪曲线的绘制,从而实现实现仿真结果的实时可视化显示。

四 数值仿真算法实现

4.1 状态空间模型建立

根据太阳翼展开动力学模型

$$\begin{aligned} x_1 &= l \\ x_2 &= \dot{l} \end{aligned} \quad (1)$$

系统状态方程写为

$$\begin{aligned} \dot{x}_1 &= x_2 \\ \dot{x}_2 &= -\frac{m_1 g + \frac{k g x_1}{\cos \theta} - \frac{k x_2^2}{\cos \theta}}{m_1 + \frac{k x_1}{\cos \theta}} + \frac{F}{m_1 + \frac{k x_1}{\cos \theta}} \end{aligned} \quad (2)$$

程序中分别封装为两个函数:

```
double C2313666yjsxDoc::F1(double x1, double x2)
{
    return x2;
}
```

```
double C2313666yjsxDoc::F2(double x1, double x2, double Fctrl)
{
    double cosTheta = cos(m_sysParams.theta * 3.14159265 /
        180.0);
    double m1 = m_sysParams.m1;
    double g = m_sysParams.g;
    double k = m_sysParams.k;

    double denom = m1 + k * x1 / cosTheta; // 等效质量
    if (denom < 0.01) denom = 0.01; // 防止除零

    // 非线性项 (修正模型: F = m g + k · g · x 在平衡时)
    double gravity_term = m1 * g + k * g * x1 / cosTheta;
```

```

    double damping_term = k * x2 * x2 / cosTheta;

    double x2_dot = -(gravity_term - damping_term) / denom +
        Fctrl / denom;
    return x2_dot;
}

```

4.2 PID 控制器实现

系统采用 PID 控制律

$$F = K_p e + K_i \int e dt + K_d \frac{de}{dt} \quad (3)$$

其中, $e = l_d - l$ 。

程序实现如下:

```
double P_term = m_pidParams.kp * error;
```

```

m_integralError += error * dt;
double I_term = m_pidParams.ki * m_integralError;

```

```

double derivative = (error - m_prevError)/dt;
double D_term = m_pidParams.kd * derivative;

```

最终控制力为:

```
output = P_term + I_term + D_term + feedforward;
```

为了补偿重力影响, 系统额外加入前馈补偿项:

```
feedforward = m1*g + k*g*m_l/cosTheta;
```

这提高了控制系统的稳态性能和响应速度。

4.3 欧拉法实现

欧拉法是一阶数值积分方法, 其离散形式为

$$x_{k+1} = x_k + hf(x_k) \quad (4)$$

程序实现如下:

```

new_x1 = x1 + dt*F1(x1,x2);
new_x2 = x2 + dt*F2(x1,x2,F_ctrl);

```

欧拉法具有算法简单, 计算速度快的优点。但这种方法的精度较低, 误差随步长增大而增大。

4.4 二阶龙格-库塔法实现

二阶龙格-库塔法 (RK2) 采用中点预测思想。

$$k_1 = f(x_n) \quad (5)$$

$$x_{mid} = x_n + \frac{h}{2}k_1 \quad (6)$$

$$k_2 = f(x_{mid}) \quad (7)$$

$$x_{n+1} = x_n + hk_2 \quad (8)$$

程序实现如下:

```
double k1_x1 = F1(x1,x2);  
double k1_x2 = F2(x1,x2,F_ctrl);  
  
double x1_mid =  
x1 + 0.5*dt*k1_x1;  
  
double x2_mid =  
x2 + 0.5*dt*k1_x2;  
  
double k2_x1 =  
F1(x1_mid,x2_mid);  
  
double k2_x2 =  
F2(x1_mid,x2_mid,F_ctrl);
```

```
new_x1 = x1 + dt*k2_x1;  
new_x2 = x2 + dt*k2_x2;
```

4.5 四阶龙格-库塔法实现

四阶龙格-库塔法 (RK4) 是工程中应用最广泛的数值积分算法。

计算公式为:

$$x_{n+1} = x_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (9)$$

程序实现如下:

```
double k1_x1 ...  
double k2_x1 ...  
double k3_x1 ...  
double k4_x1 ...
```

```
new_x1 = x1 + (dt/6.0) * (k1_x1 + 2*k2_x1 + 2*k3_x1 + k4_x1);  
new_x2 = x2 + (dt/6.0) * (k1_x2 + 2*k2_x2 + 2*k3_x2 + k4_x2);
```

四阶龙格-库塔法在仿真中数值稳定性高，结果最准确。

4.6 仿真状态更新

完成数值积分后，系统更新状态变量：

```
m_l = new_x1;  
m_l_dot = new_x2;  
m_currentTime += dt;
```

随后将新的位移、速度、误差以及控制力写入历史数据缓存，并通过 Timer 驱动实时刷新三幅曲线和右侧动画窗口，实现太阳翼展开过程的动态可视化仿真。